

1. PROJECT TITLE

AI-Driven Healthcare Anomaly Detection System for Real-Time Patient Monitoring

2. PROJECT DESCRIPTION

System Overview

The AI-Driven Healthcare Anomaly Detection System is a real-time monitoring platform designed to detect abnormal patterns in patient vital signs using machine learning techniques. The system continuously processes physiological data such as heart rate, oxygen saturation (SpO₂), body temperature, and blood pressure to identify potential health risks.

Problem Domain

Traditional healthcare monitoring systems rely on **fixed threshold values or manual monitoring**, which can delay the detection of critical health conditions. This project addresses the problem by using **machine learning algorithms to automatically detect anomalies in real time**, enabling early intervention.

Core Technology Used

- Python
- Machine Learning Models (Autoencoder, Isolation Forest)
- Apache Kafka (Data Streaming)
- Flask (Backend & Dashboard)
- PostgreSQL (Database)
- Data Visualization Libraries

Key Capabilities

- Real-time vital sign monitoring
- Machine learning based anomaly detection
- Automated health risk classification
- Interactive dashboard visualization
- Database storage of patient data
- Early warning alert generation

Target Users

- Hospitals and healthcare centers
- Medical administrators
- Healthcare researchers
- Remote patient monitoring systems

Real-world Relevance

The system can assist healthcare professionals in **early detection of abnormal patient conditions**, improving response time and patient safety in hospitals or remote monitoring setups.

3. APPLICATION SCENARIOS / USE CASES

Hospital Monitoring Systems

Hospitals can use the system to monitor patient vitals in real time and detect abnormal patterns automatically.

Remote Patient Monitoring

Patients with chronic diseases can be monitored remotely using wearable devices connected to the system.

Healthcare Research

Researchers can analyze patient vital data to identify patterns and improve predictive healthcare models.

Smart Healthcare Infrastructure

The system can be integrated with **smart hospital systems** for automated patient monitoring.

4. TECHNICAL ARCHITECTURE OVERVIEW

High-Level Architecture

System Components:

1. Data Source (Patient Vitals)
2. Data Streaming Layer (Kafka)
3. Machine Learning Processing Layer
4. Backend API (Flask)
5. Database Storage (PostgreSQL)
6. Dashboard Visualization

Data Flow

1. Patient vitals are collected from data sources.
2. Data is streamed using Kafka.
3. Machine learning models analyze the data.
4. Anomaly scores are generated.
5. Results are stored in PostgreSQL.
6. Dashboard displays real-time analytics.

Layer Explanation

Frontend Layer

- Flask-based dashboard
- Displays patient vitals and anomaly alerts

Backend Layer

- Python Flask API
- Handles processing and communication between modules

Database Layer

- PostgreSQL
- Stores patient records and anomaly results

External Services

- Apache Kafka for data streaming

Deployment Environment

- Local development environment or cloud server

5. PREREQUISITES

5.1 Software Requirements

- Python 3.x
- Visual Studio Code / PyCharm
- PostgreSQL Database
- Apache Kafka
- Flask Framework

5.2 Libraries / Dependencies

- Flask
- Pandas
- NumPy
- Scikit-learn
- TensorFlow / PyTorch
- Matplotlib / Plotly
- Kafka Python Client

5.3 Hardware Requirements

- Minimum 8GB RAM
- Multi-core processor
- Stable internet connection

6. PRIOR KNOWLEDGE REQUIRED

To implement the system, the following concepts are helpful:

- Python programming
- Machine learning fundamentals
- Flask web framework basics
- Database management (SQL)
- Data visualization concepts
- Basic networking and API concepts

7. PROJECT OBJECTIVES

Technical Objectives

- Develop a real-time healthcare monitoring system.
- Implement machine learning models for anomaly detection.

Performance Objectives

- Detect abnormal patterns with minimal delay.
- Ensure efficient processing of streaming data.

Deployment Objectives

- Deploy a dashboard for real-time monitoring.

Learning Outcomes

- Understanding real-time data streaming
- Implementing anomaly detection models
- Building a backend system with Flask
- Designing healthcare analytics dashboards

8. SYSTEM WORKFLOW

1. User accesses the monitoring dashboard.
2. Patient vital data is collected and streamed.
3. Data preprocessing is performed.
4. Machine learning models analyze the data.
5. Anomaly detection results are generated.
6. Results are stored in the database.

7. Dashboard displays alerts and visualizations.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

Problem Definition

Manual monitoring of patient vitals is inefficient and prone to delays. The system aims to automate anomaly detection using machine learning.

Functional Requirements

- Data collection and ingestion
- Data preprocessing
- Anomaly detection
- Dashboard visualization
- Database storage

Non-Functional Requirements

- System reliability
- Data security
- Low response time
- Scalability

Technology Stack Justification

Component	Technology
Backend	Flask (Python)
ML Models	Isolation Forest, Autoencoder
Database	PostgreSQL
Streaming	Apache Kafka
Visualization	Plotly / Matplotlib

10. MILESTONE 2: ENVIRONMENT SETUP

Development Setup

- Install Python
- Install PostgreSQL
- Install Kafka

Dependency Installation

Example command:

pip install flask pandas numpy scikit-learn matplotlib kafka-python

Project Structure Creation

Create folders for source code, database scripts, and configuration files.

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

Feature 1 – Data Collection

Patient vital data is collected and passed to the system.

Feature 2 – Data Processing

Data preprocessing removes missing values and prepares data for ML models.

Feature 3 – Anomaly Detection

Machine learning algorithms analyze patient vitals and calculate anomaly scores.

Feature 4 – Dashboard Visualization

Results are displayed using charts and alerts.

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

Component Integration

All modules including data ingestion, ML processing, and dashboard are integrated.

Performance Optimization

Data pipelines optimized for faster processing.

Security Enhancements

Database access controls implemented.

Error Handling

Validation checks for missing or incorrect data.

13. MILESTONE 5: TESTING & VALIDATION

Test Case ID	Input	Expected Output	Status
TC01	Patient vital data	Anomaly score generated	Verified
TC02	Valid dataset	Dashboard visualization	Verified
TC03	Abnormal data	Alert generation	Verified

Performance Metrics

- Response time: < 3 seconds
- Stable dashboard rendering

- Successful anomaly detection execution

14. DEPLOYMENT

Deployment Architecture

System deployed on local server for demonstration.

Deployment Steps

1. Install dependencies
2. Configure database
3. Start backend server
4. Launch dashboard

15. PROJECT STRUCTURE

Rutujjs Initial clean commit		226b336 · yesterday	🕒 1 Commit
📁 api	Initial clean commit	yesterday	
📁 config	Initial clean commit	yesterday	
📁 data	Initial clean commit	yesterday	
📁 database	Initial clean commit	yesterday	
📁 frontend/templates	Initial clean commit	yesterday	
📁 kafka	Initial clean commit	yesterday	
📁 models	Initial clean commit	yesterday	
📁 notifications	Initial clean commit	yesterday	
📁 scripts	Initial clean commit	yesterday	
📄 README.md	Initial clean commit	yesterday	
📄 requirements.txt	Initial clean commit	yesterday	
📄 setup_db.py	Initial clean commit	yesterday	

Explanation:

- api/
Contains the backend API endpoints that handle communication between the frontend dashboard and the backend processing system.
- config/
Stores configuration settings such as database connections, Kafka settings, and environment variables.
- data/
Holds the datasets and patient vital data used for processing and anomaly detection.
- database/
Contains files related to database setup, schema creation, and data storage using PostgreSQL.
- frontend/templates/

- Stores the HTML templates for the dashboard UI, which display patient data, charts, and alerts.
- kafka/
Implements real-time data streaming using Apache Kafka to process incoming patient vital data.
- models/
Contains the machine learning models (e.g., Isolation Forest, Autoencoder) used to detect anomalies in patient vitals.
- notifications/
Handles the alert and notification system, sending warnings when abnormal health patterns are detected.
- scripts/
Includes utility scripts for tasks like data preprocessing, model training, or system automation.
- README.md
Provides documentation of the project, including description, setup instructions, and usage guidelines.
- requirements.txt
Lists all Python dependencies and libraries required to run the project.
- setup_db.py
Script used to initialize and configure the PostgreSQL database, including table creation.

16. RESULTS

The system successfully detected abnormal patterns in patient vital data and displayed alerts on the dashboard.

Key Results:

- Real-time anomaly detection achieved
- Dashboard visualization implemented
- Data stored successfully in database

17. ADVANTAGES & LIMITATIONS

Advantages

- Real-time monitoring
- Automated anomaly detection
- Scalable architecture

Limitations

- Requires reliable data input
- Model accuracy depends on training data

18. FUTURE ENHANCEMENTS

- Integration with wearable devices

- Cloud deployment
- Mobile application interface
- Advanced deep learning models
- Automated healthcare alerts

19. CONCLUSION

The AI-Driven Healthcare Anomaly Detection System demonstrates the integration of machine learning, data streaming, and web technologies to build an intelligent healthcare monitoring platform. The system successfully detects abnormal patterns in patient vital signs and provides real-time insights through a dashboard interface. This project highlights the potential of AI-driven systems in improving healthcare monitoring and patient safety.

20. APPENDIX:

- Source Code
- Configuration Files

Rutujjs Initial clean commit		228b336 · yesterday	🕒 1 Commit
api	Initial clean commit	yesterday	
config	Initial clean commit	yesterday	
data	Initial clean commit	yesterday	
database	Initial clean commit	yesterday	
frontend/templates	Initial clean commit	yesterday	
kafka	Initial clean commit	yesterday	
models	Initial clean commit	yesterday	
notifications	Initial clean commit	yesterday	
scripts	Initial clean commit	yesterday	
README.md	Initial clean commit	yesterday	
requirements.txt	Initial clean commit	yesterday	
setup_db.py	Initial clean commit	yesterday	

- GitHub Repository Link --
https://github.com/Rutujjs/GEN_AI_SMARTBRIDGE

- Live Demo Link--
https://drive.google.com/drive/folders/1y-Rb64aJyC2EvGYTlayCAocWjPiYr_Cw?usp=drive_link